

AHDS - PYTHON-01

Python Programming & Engineering Reference

PART 2 - TYPES, COLLECTIONS, TEXT & BUILT-INS

Chapters 21-67 • English / Français

Engineering focus / Orientation ingénierie

Data representation, text and binary processing, collection semantics, iteration and built-in tools for reliable engineering software.
Représentation des données, traitement du texte et du binaire, sémantique des collections, itération et outils intégrés pour des logiciels d’ingénierie fiables.

Editorial note / Note éditoriale
Python 3.15.0 was released on 9 October 2026. Features introduced in Python 3.15 are explicitly version-dependent; examples must be run with a compatible interpreter.
Python 3.15.0 est sorti le 9 octobre 2026. Les fonctionnalités introduites avec Python 3.15 dépendent de la version utilisée ; les exemples nécessitent un interpréteur compatible.



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.

Table des matières / Contents

Section A — Type System & Numbers / Système de types et nombres

21. Python Type System Fundamentals / Fondamentaux du système de types Python
22. Boolean Type — bool / Type booléen — bool
23. Integer Type — int / Type entier — int
24. Floating-Point Type — float / Type à virgule flottante — float
25. Complex Numbers — complex / Nombres complexes — complex
26. Numeric Conversion & Representation / Conversion et représentation numériques
27. Rounding, Precision & Numeric Limits / Arrondi, précision et limites numériques
28. Decimal & Exact Decimal Arithmetic — decimal / Arithmétique décimale exacte — decimal
29. Fractions & Rational Arithmetic — fractions / Fractions et arithmétique rationnelle — fractions

Section B — Strings & Text Processing / Chaînes et traitement de texte

30. Strings — str / Chaînes de caractères — str
31. String Methods & Text Processing / Méthodes de chaînes et traitement de texte
32. String Formatting / Formatage des chaînes
33. Formatted String Literals — f-strings / Littéraux de chaîne formatés — f-strings
34. Template Strings & Modern String Features / Chaînes modèles et fonctions modernes
35. Unicode, Characters & Text Encoding / Unicode, caractères et encodage du texte
36. Regular Expressions Fundamentals — re / Fondamentaux des expressions régulières — re
37. Regex Patterns, Character Classes & Quantifiers / Motifs regex, classes de caractères et quantificateurs
38. Regex Groups, Captures, Lookaround & Substitution / Groupes regex, captures, lookaround et substitution
39. Regular Expressions for Engineering Logs, Responses & Data / Expressions régulières pour journaux, réponses et données d'ingénierie

Section C — Binary Data / Données binaires

40. bytes / bytes
41. bytearray / bytearray
42. memoryview & Buffer-Oriented Data / memoryview et données orientées tampon

Section D — Sequences / Séquences

43. Lists — list / Listes — list
44. Tuples — tuple / Tuples — tuple
45. Ranges — range / Plages — range
46. Sequence Operations / Opérations sur les séquences
47. Slicing in Depth / Découpage avancé — slicing
48. Packing, Unpacking & Starred Expressions / Empaquetage, dépaquetage et expressions étoilées

Section E — Mappings & Sets / Mappings et ensembles

49. Dictionaries — dict / Dictionnaires — dict
50. Dictionary Views & Advanced Mapping Operations / Vues de dictionnaire et opérations avancées
51. Immutable Mapping — frozendict / Mapping immuable — frozendict
52. Sets — set / Ensembles — set
53. Frozen Sets — frozenset / Ensembles immuables — frozenset
54. Set Algebra / Algèbre des ensembles

Section F — Comprehensions & Iteration / Compréhensions et itération

55. List Comprehensions / Compréhensions de listes
56. Set & Dictionary Comprehensions / Compréhensions d'ensembles et de dictionnaires
57. Nested Comprehensions & Unpacking / Compréhensions imbriquées et dépaquetage
58. Iterable, Iterator & Iteration Fundamentals / Fondamentaux des itérables, itérateurs et de l'itération

Section G — Object/Data Semantics / Sémantique des objets et données

59. Mutability & Immutability / Mutabilité et immutabilité
60. Copying Objects / Copie des objets



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



61. Hashability / Hachabilité
62. Truth-Value Testing / Évaluation de la valeur de vérité
63. None, Sentinel Values & Missing-Value Design / None, sentinelles et conception des valeurs manquantes

Section H — Specialized Standard Collections & Algorithms / Collections et algorithmes spécialisés

64. Specialized Standard Collections & Algorithms / Collections et algorithmes standard spécialisés

Section I — Built-in Functions / Fonctions intégrées

65. Built-in Functions — Numeric, Conversion & Representation / Fonctions intégrées — numérique, conversion et représentation
66. Built-in Functions — Collections, Iteration & Inspection / Fonctions intégrées — collections, itération et inspection
67. Built-in Functions — Runtime, Objects & Dynamic Operations / Fonctions intégrées — exécution, objets et opérations dynamiques



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Section A — Type System & Numbers / Système de types et nombres

Chapter 21 — Python Type System Fundamentals

Chapitre 21 — Fondamentaux du système de types Python

ENGLISH	FRANÇAIS
What is the core idea of Python Type System Fundamentals? Python uses dynamic, strong typing: names bind to objects, and every object has a runtime type. Type relationships affect operators, protocols and APIs.	**Quelle est l'idée essentielle de « Fondamentaux du système de types Python » ?** Ce chapitre présente fondamentaux du système de types python. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 21, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 21, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
raw = b"\x10\x27"  # 0x2710, big-endian
reading = int.from_bytes(raw, "big")
assert isinstance(reading, int) and reading == 10000
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 22 — Boolean Type — bool

Chapitre 22 — Type booléen — bool

ENGLISH	FRANÇAIS
What is the core idea of Boolean Type — bool? bool has two singleton values, True and False, and participates in truth-value testing; it is also a subclass of int.	**Quelle est l'idée essentielle de « Type booléen — bool » ?** Ce chapitre présente type booléen — bool. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 22, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 22, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
enabled = bool(status_flags & 0x01)
if enabled:
    start_measurement()
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 23 — Integer Type — int

Chapitre 23 — Type entier — int

ENGLISH	FRANÇAIS
What is the core idea of Integer Type — int? int represents arbitrary-precision integers, making it suitable for counters, masks, identifiers and exact integer arithmetic.	**Quelle est l'idée essentielle de « Type entier — int » ?** Ce chapitre présente type entier — int. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 23, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 23, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
mask = 0b1111
channel = raw_status & mask
packet_id = int("1A2F", 16)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 24 — Floating-Point Type — float

Chapitre 24 — Type à virgule flottante — float

ENGLISH	FRANÇAIS
What is the core idea of Floating-Point Type — float? float is normally IEEE 754 binary64; many decimal fractions are not exactly representable, so comparisons require engineering judgement.	**Quelle est l'idée essentielle de « Type à virgule flottante — float » ?** Ce chapitre présente type à virgule flottante — float. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 24, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 24, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
import math
measured = 0.1 + 0.2
assert math.isclose(measured, 0.3, rel_tol=1e-9)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 25 — Complex Numbers — complex

Chapitre 25 — Nombres complexes — complex

ENGLISH	FRANÇAIS
What is the core idea of Complex Numbers — complex? complex stores real and imaginary floating-point components and is useful in signal, impedance and frequency-domain calculations.	**Quelle est l'idée essentielle de « Nombres complexes — complex » ?** Ce chapitre présente nombres complexes — complex. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 25, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 25, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
z = complex(12.0, -5.0)  # measured impedance, ohms
magnitude = abs(z)
assert magnitude == 13.0
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 26 — Numeric Conversion & Representation

Chapitre 26 — Conversion et représentation numériques

ENGLISH	FRANÇAIS
What is the core idea of Numeric Conversion & Representation? Conversion functions such as int(), float(), complex(), bin(), oct(), hex() and format() bridge numeric values and external representations.	**Quelle est l'idée essentielle de « Conversion et représentation numériques » ?** Ce chapitre présente conversion et représentation numériques. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 26, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 26, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
raw_adc = "0x0FFF"
counts = int(raw_adc, 16)
voltage = counts * 3.0 / 4095
assert counts == 4095 and voltage == 3.0
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 27 — Rounding, Precision & Numeric Limits

Chapitre 27 — Arrondi, précision et limites numériques

ENGLISH	FRANÇAIS
What is the core idea of Rounding, Precision & Numeric Limits? rounding and finite precision affect measurements and thresholds; sys.float_info and math functions help reason about numeric limits.	**Quelle est l'idée essentielle de « Arrondi, précision et limites numériques » ?** Ce chapitre présente arrondi, précision et limites numériques. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 27, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 27, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
import math, sys
threshold = 2.5
measured = math.nextafter(threshold, math.inf)
assert measured > threshold
print(sys.float_info.epsilon, math.ulp(threshold))
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 28 — Decimal & Exact Decimal Arithmetic — decimal

Chapitre 28 — Arithmétique décimale exacte — decimal

ENGLISH	FRANÇAIS
What is the core idea of Decimal & Exact Decimal Arithmetic — decimal? decimal.Decimal provides base-10 arithmetic with configurable precision and rounding, useful where exact decimal semantics matter.	**Quelle est l'idée essentielle de « Arithmétique décimale exacte — decimal » ?** Ce chapitre présente arithmétique décimale exacte — decimal. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 28, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 28, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
from decimal import Decimal
voltage = Decimal("24.10")
limit = Decimal("24.00")
print(voltage > limit)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 29 — Fractions & Rational Arithmetic — fractions

Chapitre 29 — Fractions et arithmétique rationnelle — fractions

ENGLISH	FRANÇAIS
What is the core idea of Fractions & Rational Arithmetic — fractions? fractions.Fraction represents rational values exactly as numerator/denominator pairs and is useful for ratios and exact scaling.	**Quelle est l'idée essentielle de « Fractions et arithmétique rationnelle — fractions » ?** Ce chapitre présente fractions et arithmétique rationnelle — fractions. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 29, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 29, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
from fractions import Fraction
ratio = Fraction(3, 8)
scaled = 4095 * ratio
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Section B — Strings & Text Processing / Chaînes et traitement de texte

Chapter 30 — Strings — str

Chapitre 30 — Chaînes de caractères — str

ENGLISH	FRANÇAIS
What is the core idea of Strings — str? str is an immutable Unicode text sequence; indexing returns characters as one-character strings.	**Quelle est l'idée essentielle de « Chaînes de caractères — str » ?** Ce chapitre présente chaînes de caractères — str. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 30, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 30, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
response = "CH1:24.18 V"
assert response.startswith("CH1:")
reading_text = response.split(":", 1)[1].strip()
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 31 — String Methods & Text Processing

Chapitre 31 — Méthodes de chaînes et traitement de texte

ENGLISH	FRANÇAIS
What is the core idea of String Methods & Text Processing? string methods provide searching, splitting, joining, normalization and validation without changing the original immutable string.	**Quelle est l'idée essentielle de « Méthodes de chaînes et traitement de texte » ?** Ce chapitre présente méthodes de chaînes et traitement de texte. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 31, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 31, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
reply = "  TEMP=23.5;STATUS=OK  "
fields = dict(item.split("=", 1) for item in reply.strip().split(";"))
assert fields["STATUS"] == "OK"
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 32 — String Formatting

Chapitre 32 — Formatage des chaînes

ENGLISH	FRANÇAIS
What is the core idea of String Formatting? format specifications control width, alignment, sign, precision and numeric bases for human-readable reports and machine-facing text.	**Quelle est l'idée essentielle de « Formatage des chaînes » ?** Ce chapitre présente formatage des chaînes. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 32, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 32, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
voltage = 24.1837
report = "VBAT={:08.3f} V".format(voltage)
assert report == "VBAT=0024.184 V"
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 33 — Formatted String Literals — f-strings

Chapitre 33 — Littéraux de chaîne formatés — f-strings

ENGLISH	FRANÇAIS
What is the core idea of Formatted String Literals — f-strings? f-strings embed expressions directly in string literals and support the format mini-language for concise, readable output.	**Quelle est l’idée essentielle de « Littéraux de chaîne formatés — f-strings » ?** Ce chapitre présente littéraux de chaîne formatés — f-strings. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d’ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d’ingénierie ?** Cela influence la justesse, l’échange des données, les performances et la maintenabilité. Dans les logiciels de test, d’automatisation et d’interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d’état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l’API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d’ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 33, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 33, documenter la représentation d’entrée attendue et le comportement en cas d’échec.

Shared engineering example — code remains English only

```
channel, voltage = 3, 3.28765
message = f"CH{channel:02d}: {voltage:.3f} V"
assert message == "CH03: 3.288 V"
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 34 — Template Strings & Modern String Features

Chapitre 34 — Chaînes modèles et fonctions modernes

ENGLISH	FRANÇAIS
What is the core idea of Template Strings & Modern String Features? template-oriented string features separate data from text structure and can reduce accidental formatting complexity in generated messages.	**Quelle est l'idée essentielle de « Chaînes modèles et fonctions modernes » ?** Ce chapitre présente chaînes modèles et fonctions modernes. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 34, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 34, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
from string import Template
t = Template("DEVICE=$device STATUS=$status")
message = t.substitute(device="ADC1", status="OK")
assert message == "DEVICE=ADC1 STATUS=OK"
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 35 — Unicode, Characters & Text Encoding

Chapitre 35 — Unicode, caractères et encodage du texte

ENGLISH	FRANÇAIS
What is the core idea of Unicode, Characters & Text Encoding? Unicode separates characters from encoded bytes; encode() and decode() define the boundary between text and binary I/O.	**Quelle est l'idée essentielle de « Unicode, caractères et encodage du texte » ?** Ce chapitre présente unicode, caractères et encodage du texte. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 35, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 35, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
reply = "TEMP=23.5°C"
payload = reply.encode("utf-8")
text = payload.decode("utf-8")
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 36 — Regular Expressions Fundamentals — re

Chapitre 36 — Fondamentaux des expressions régulières — re

ENGLISH	FRANÇAIS
What is the core idea of Regular Expressions Fundamentals — re? the re module matches text using regular-expression patterns and is effective for structured but not fully fixed input.	**Quelle est l'idée essentielle de « Fondamentaux des expressions régulières — re » ?** Ce chapitre présente fondamentaux des expressions régulières — re. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 36, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 36, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
import re
m = re.search(r"TEMP=(-?\d+(?:\.\d+)?)", "TEMP=23.5 C")
temp_c = float(m.group(1)) if m else None
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 37 — Regex Patterns, Character Classes & Quantifiers

Chapitre 37 — Motifs regex, classes de caractères et quantificateurs

ENGLISH	FRANÇAIS
What is the core idea of Regex Patterns, Character Classes & Quantifiers? character classes, anchors and quantifiers describe repeated or constrained text while controlling how much input is consumed.	**Quelle est l'idée essentielle de « Motifs regex, classes de caractères et quantificateurs » ?** Ce chapitre présente motifs regex, classes de caractères et quantificateurs. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 37, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 37, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
import re
packet = "CH12=3.28;CH2=1.20"
readings = re.findall(r"CH(\d{1,2})=([0-9]+\.[0-9]{2})", packet)
assert len(readings) == 2
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 38 — Regex Groups, Captures, Lookaround & Substitution

Chapitre 38 — Groupes regex, captures, lookaround et substitution

ENGLISH	FRANÇAIS
What is the core idea of Regex Groups, Captures, Lookaround & Substitution? groups capture subexpressions; lookaround tests context without consuming it; substitution transforms matched text.	**Quelle est l'idée essentielle de « Groupes regex, captures, lookaround et substitution » ?** Ce chapitre présente groupes regex, captures, lookaround et substitution. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 38, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 38, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
import re
line = "ADC: CH07 = 2.48 V"
m = re.search(r"(?<=CH)(\d{2})\s*=\s*([0-9.]+)", line)
assert m and m.groups() == ("07", "2.48")
clean = re.sub(r"\s*=\s*", "=", line)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 39 — Regular Expressions for Engineering Logs, Responses & Data

Chapitre 39 — Expressions régulières pour journaux, réponses et données d’ingénierie

ENGLISH	FRANÇAIS
What is the core idea of Regular Expressions for Engineering Logs, Responses & Data? regex can extract timestamps, IDs, measurements and status fields from instrument responses and engineering logs when formats are semi-structured.	**Quelle est l’idée essentielle de « Expressions régulières pour journaux, réponses et données d’ingénierie » ?** Ce chapitre présente expressions régulières pour journaux, réponses et données d’ingénierie. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d’ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d’ingénierie ?** Cela influence la justesse, l’échange des données, les performances et la maintenabilité. Dans les logiciels de test, d’automatisation et d’interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d’état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l’API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d’ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 39, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 39, documenter la représentation d’entrée attendue et le comportement en cas d’échec.

Shared engineering example — code remains English only

```
import re
line = "12:41:03 ID=26012601 VBAT=24.18 STATUS=OK"
m = re.search(r"ID=(\d+)\s+VBAT=([0-9.]+)\s+STATUS=(\w+)", line)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Section C — Binary Data / Données binaires

Chapter 40 — bytes

Chapitre 40 — bytes

ENGLISH	FRANÇAIS
What is the core idea of bytes? bytes is an immutable sequence of integers 0–255 and is the natural representation for received binary packets and encoded text.	**Quelle est l'idée essentielle de « bytes » ?** Ce chapitre présente bytes. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 40, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 40, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
frame = bytes([0xAA, 0x01, 0x10, 0x55])
header = frame[0]
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 41 — bytearray

Chapitre 41 — bytearray

ENGLISH	FRANÇAIS
What is the core idea of bytearray? bytearray is a mutable byte buffer suited to packet construction, in-place field updates and binary protocol assembly.	**Quelle est l'idée essentielle de « bytearray » ?** Ce chapitre présente bytearray. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 41, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 41, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
frame = bytearray(8)
frame[0:2] = b"\xAA\x55"
frame[2] = 3
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 42 — memoryview & Buffer-Oriented Data

Chapitre 42 — memoryview et données orientées tampon

ENGLISH	FRANÇAIS
What is the core idea of memoryview & Buffer-Oriented Data? memoryview exposes buffer-protocol data without copying, enabling efficient slicing and binary processing of large buffers.	**Quelle est l'idée essentielle de « memoryview et données orientées tampon » ?** Ce chapitre présente memoryview et données orientées tampon. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 42, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 42, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
buffer = bytearray(4096)
view = memoryview(buffer)
header = view[:16]  # no copy
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Section D — Sequences / Séquences

Chapter 43 — Lists — list

Chapitre 43 — Listes — list

ENGLISH	FRANÇAIS
What is the core idea of Lists — list? list is a mutable ordered sequence optimized for general-purpose storage, indexing, appending and iteration.	**Quelle est l'idée essentielle de « Listes — list » ?** Ce chapitre présente listes — list. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 43, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 43, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
samples = [1.02, 1.01, 1.03]
samples.append(1.00)
avg = sum(samples) / len(samples)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 44 — Tuples — tuple

Chapitre 44 — Tuples — tuple

ENGLISH	FRANÇAIS
What is the core idea of Tuples — tuple? tuple is an immutable ordered sequence often used for fixed records, coordinates, return values and hashable composite keys.	**Quelle est l’idée essentielle de « Tuples — tuple » ?** Ce chapitre présente tuples — tuple. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d’ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d’ingénierie ?** Cela influence la justesse, l’échange des données, les performances et la maintenabilité. Dans les logiciels de test, d’automatisation et d’interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d’état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l’API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d’ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 44, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 44, documenter la représentation d’entrée attendue et le comportement en cas d’échec.

Shared engineering example — code remains English only

```
point = (12.5, -3.0)
x, y = point
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 45 — Ranges — range

Chapitre 45 — Plages — range

ENGLISH	FRANÇAIS
What is the core idea of Ranges — range? range represents arithmetic integer progressions compactly without storing every value.	**Quelle est l'idée essentielle de « Plages — range » ?** Ce chapitre présente plages — range. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 45, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 45, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
for channel in range(1, 17):
    poll_channel(channel)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 46 — Sequence Operations

Chapitre 46 — Opérations sur les séquences

ENGLISH	FRANÇAIS
What is the core idea of Sequence Operations? sequence protocols provide indexing, length, membership, concatenation, repetition and iteration across compatible sequence types.	**Quelle est l'idée essentielle de « Opérations sur les séquences » ?** Ce chapitre présente opérations sur les séquences. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 46, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 46, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
samples = [100, 101, 99, 102]
assert 101 in samples and len(samples) == 4
shifted = samples[1:] + samples[:1]
assert shifted == [101, 99, 102, 100]
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 47 — Slicing in Depth

Chapitre 47 — Découpage avancé — slicing

ENGLISH	FRANÇAIS
What is the core idea of Slicing in Depth? slicing selects ranges using start:stop:step and can express copies, windows, reversals and strided access.	**Quelle est l'idée essentielle de « Découpage avancé — slicing » ?** Ce chapitre présente découpage avancé — slicing. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 47, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 47, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
window = samples[100:200]
reversed_data = samples[::-1]
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 48 — Packing, Unpacking & Starred Expressions

Chapitre 48 — Empaquetage, dépaquetage et expressions étoilées

ENGLISH	FRANÇAIS
What is the core idea of Packing, Unpacking & Starred Expressions? packing and unpacking map multiple values to structured targets; starred expressions absorb or expand variable-length portions.	**Quelle est l'idée essentielle de « Empaquetage, dépaquetage et expressions étoilées » ?** Ce chapitre présente empaquetage, dépaquetage et expressions étoilées. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 48, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 48, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
status, voltage, current = response
first, *middle, last = values
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Section E — Mappings & Sets / Mappings et ensembles

Chapter 49 — Dictionaries — dict

Chapitre 49 — Dictionnaires — dict

ENGLISH	FRANÇAIS
What is the core idea of Dictionaries — dict? dict maps hashable keys to values and preserves insertion order, making it central to configuration, records and lookup tables.	**Quelle est l'idée essentielle de « Dictionnaires — dict » ?** Ce chapitre présente dictionnaires — dict. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 49, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 49, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
limits = {"vbat_min": 21.0, "vbat_max": 29.0}
if voltage > limits["vbat_max"]:
    raise ValueError("Overvoltage")
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 50 — Dictionary Views & Advanced Mapping Operations

Chapitre 50 — Vues de dictionnaire et opérations avancées

ENGLISH	FRANÇAIS
What is the core idea of Dictionary Views & Advanced Mapping Operations? dictionary views are dynamic windows over keys, values and items; advanced operations support merging, iteration and set-like key analysis.	**Quelle est l'idée essentielle de « Vues de dictionnaire et opérations avancées » ?** Ce chapitre présente vues de dictionnaire et opérations avancées. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 50, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 50, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
limits = {"ADC": 4095, "DAC": 4095}
channels = limits.keys()  # dynamic dictionary view
limits["PWM"] = 1023
assert "PWM" in channels
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 51 — Immutable Mapping — frozendict

Chapitre 51 — Mapping immuable — frozendict

ENGLISH	FRANÇAIS
What is the core idea of Immutable Mapping — frozendict? Starting with Python 3.15, frozendict is a built-in immutable mapping. It preserves insertion order, cannot be changed after creation, and is hashable only when all keys and values are hashable. It is not a dict subclass. Older Python versions do not provide this built-in type. frozendict is an immutable mapping concept; availability depends on the Python version/API in use, so portability must be verified.	Quelle est l'idée essentielle de « Mapping immuable — frozendict » ? Depuis Python 3.15, frozendict est un mapping immuable intégré. Il conserve l'ordre d'insertion, interdit les modifications et n'est hachable que si toutes les clés et valeurs le sont. Ce n'est pas une sous-classe de dict. Les anciennes versions ne proposent pas ce type intégré. Ce chapitre présente mapping immuable — frozendict. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 51, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 51, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
import sys
if sys.version_info >= (3, 15):
    limits = frozendict(vbat_max=29.0, ibat_max=5.0)
    assert limits["vbat_max"] == 29.0
    # Item assignment raises TypeError.
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 52 — Sets — set

Chapitre 52 — Ensembles — set

ENGLISH	FRANÇAIS
What is the core idea of Sets — set? set stores unique hashable elements and supports fast membership testing and mathematical set operations.	**Quelle est l’idée essentielle de « Ensembles — set » ?** Ce chapitre présente ensembles — set. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d’ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d’ingénierie ?** Cela influence la justesse, l’échange des données, les performances et la maintenabilité. Dans les logiciels de test, d’automatisation et d’interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d’état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l’API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d’ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 52, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 52, documenter la représentation d’entrée attendue et le comportement en cas d’échec.

Shared engineering example — code remains English only

```
expected = {"VBAT", "IBAT", "TEMP"}
received = set(fields)
missing = expected - received
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 53 — Frozen Sets — frozenset

Chapitre 53 — Ensembles immuables — frozenset

ENGLISH	FRANÇAIS
What is the core idea of Frozen Sets — frozenset? frozenset is an immutable, hashable set suitable for constants, keys and cached state descriptions.	**Quelle est l'idée essentielle de « Ensembles immuables — frozenset » ?** Ce chapitre présente ensembles immuables — frozenset. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 53, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 53, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
enabled = frozenset({"UART", "SPI"})
requested = {"SPI", "I2C"}
assert requested & enabled == {"SPI"}
assert "UART" in enabled
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 54 — Set Algebra

Chapitre 54 — Algèbre des ensembles

ENGLISH	FRANÇAIS
What is the core idea of Set Algebra? union, intersection, difference and symmetric difference provide concise reasoning about capabilities, flags, test sets and inventories.	**Quelle est l'idée essentielle de « Algèbre des ensembles » ?** Ce chapitre présente algèbre des ensembles. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 54, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 54, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
missing = required - received
extra = received - required
valid = not missing
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Section F — Comprehensions & Iteration / Compréhensions et itération

Chapter 55 — List Comprehensions

Chapitre 55 — Compréhensions de listes

ENGLISH	FRANÇAIS
What is the core idea of List Comprehensions? list comprehensions construct lists from iteration plus optional filtering in a compact expression.	**Quelle est l'idée essentielle de « Compréhensions de listes » ?** Ce chapitre présente compréhensions de listes. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 55, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 55, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
valid = [x for x in samples if 0.0 <= x <= 5.0]
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 56 — Set & Dictionary Comprehensions

Chapitre 56 — Compréhensions d’ensembles et de dictionnaires

ENGLISH	FRANÇAIS
What is the core idea of Set & Dictionary Comprehensions? set and dictionary comprehensions build unique collections or mappings directly from iterable data.	**Quelle est l’idée essentielle de « Compréhensions d’ensembles et de dictionnaires » ?** Ce chapitre présente compréhensions d’ensembles et de dictionnaires. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d’ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d’ingénierie ?** Cela influence la justesse, l’échange des données, les performances et la maintenabilité. Dans les logiciels de test, d’automatisation et d’interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d’état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l’API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d’ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 56, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 56, documenter la représentation d’entrée attendue et le comportement en cas d’échec.

Shared engineering example — code remains English only

```
channels = {name: value for name, value in readings if value is not None}
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 57 — Nested Comprehensions & Unpacking

Chapitre 57 — Compréhensions imbriquées et dépaquetage

ENGLISH	FRANÇAIS
What is the core idea of Nested Comprehensions & Unpacking? nested comprehensions express multi-dimensional transformations; unpacking can simplify structured source records.	**Quelle est l'idée essentielle de « Compréhensions imbriquées et dépaquetage » ?** Ce chapitre présente compréhensions imbriquées et dépaquetage. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 57, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 57, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
frames = [[1, 2], [3, 4]]
flat = [sample for frame in frames for sample in frame]
assert flat == [1, 2, 3, 4]
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 58 — Iterable, Iterator & Iteration Fundamentals

Chapitre 58 — Fondamentaux des itérables, itérateurs et de l’itération

ENGLISH	FRANÇAIS
What is the core idea of Iterable, Iterator & Iteration Fundamentals? an iterable can produce an iterator; an iterator maintains traversal state and yields items through __next__().	**Quelle est l’idée essentielle de « Fondamentaux des itérables, itérateurs et de l’itération » ?** Ce chapitre présente fondamentaux des itérables, itérateurs et de l’itération. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d’ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d’ingénierie ?** Cela influence la justesse, l’échange des données, les performances et la maintenabilité. Dans les logiciels de test, d’automatisation et d’interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d’état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l’API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d’ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 58, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 58, documenter la représentation d’entrée attendue et le comportement en cas d’échec.

Shared engineering example — code remains English only

```
it = iter(samples)
first = next(it, None)
for value in it:
    process(value)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Section G — Object/Data Semantics / Sémantique des objets et données

Chapter 59 — Mutability & Immutability

Chapitre 59 — Mutabilité et immutabilité

ENGLISH	FRANÇAIS
What is the core idea of Mutability & Immutability? mutable objects can change in place while immutable objects cannot; this distinction affects aliasing, APIs and shared state.	**Quelle est l'idée essentielle de « Mutabilité et immutabilité » ?** Ce chapitre présente mutabilité et immutabilité. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 59, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 59, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
config = {"channels": [1, 2]}
alias = config
alias["channels"].append(3)
assert config["channels"] == [1, 2, 3]
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 60 — Copying Objects

Chapitre 60 — Copie des objets

ENGLISH	FRANÇAIS
What is the core idea of Copying Objects? assignment does not copy objects; shallow and deep copying have different behavior for nested mutable structures.	**Quelle est l'idée essentielle de « Copie des objets » ?** Ce chapitre présente copie des objets. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 60, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 60, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
from copy import deepcopy
config_copy = deepcopy(config)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 61 — Hashability

Chapitre 61 — Hachabilité

ENGLISH	FRANÇAIS
What is the core idea of Hashability? hashable objects have a stable hash/equality contract and may be used as dictionary keys or set members.	**Quelle est l'idée essentielle de « Hachabilité » ?** Ce chapitre présente hachabilité. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 61, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 61, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
device_keys = {(0x1234, 0x5678): "USB adapter"}
assert device_keys[(0x1234, 0x5678)] == "USB adapter"
# Lists cannot be dictionary keys: they are unhashable.
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 62 — Truth-Value Testing

Chapitre 62 — Évaluation de la valeur de vérité

ENGLISH	FRANÇAIS
What is the core idea of Truth-Value Testing? objects participate in Boolean contexts through standard truth-value rules, __bool__() or __len__().	**Quelle est l'idée essentielle de « Évaluation de la valeur de vérité » ?** Ce chapitre présente évaluation de la valeur de vérité. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 62, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 62, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
samples = []
if not samples:
    status = "NO_DATA"
else:
    status = "READY"
assert status == "NO_DATA"
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 63 — None, Sentinel Values & Missing-Value Design

Chapitre 63 — None, sentinelles et conception des valeurs manquantes

ENGLISH	FRANÇAIS
What is the core idea of None, Sentinel Values & Missing-Value Design? None represents absence, while a unique sentinel distinguishes “not supplied” from valid values such as None, 0 or an empty string.	**Quelle est l’idée essentielle de « None, sentinelles et conception des valeurs manquantes » ?** Ce chapitre présente none, sentinelles et conception des valeurs manquantes. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d’ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d’ingénierie ?** Cela influence la justesse, l’échange des données, les performances et la maintenabilité. Dans les logiciels de test, d’automatisation et d’interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d’état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l’API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d’ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 63, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 63, documenter la représentation d’entrée attendue et le comportement en cas d’échec.

Shared engineering example — code remains English only

```
MISSING = object()
value = config.get("timeout", MISSING)
if value is MISSING:
    value = DEFAULT_TIMEOUT
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Section H — Specialized Standard Collections & Algorithms / Collections et algorithmes spécialisés

Chapter 64 — Specialized Standard Collections & Algorithms

Chapitre 64 — Collections et algorithmes standard spécialisés

ENGLISH	FRANÇAIS
What is the core idea of Specialized Standard Collections & Algorithms? heapq provides priority queues, bisect maintains sorted-list insertion/search, and graphlib supports dependency ordering; specialized collections should be chosen only when their semantics match the problem.	**Quelle est l'idée essentielle de « Collections et algorithmes standard spécialisés » ?** Ce chapitre présente collections et algorithmes standard spécialisés. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 64, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 64, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
import heapq
from bisect import insort
from graphlib import TopologicalSorter
queue = []
heapq.heappush(queue, (2, "low priority"))
ordered = [1, 4, 9]
insort(ordered, 6)
order = tuple(TopologicalSorter({"test": {"init"}}).static_order())
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Section I — Built-in Functions / Fonctions intégrées

Chapter 65 — Built-in Functions — Numeric, Conversion & Representation

Chapitre 65 — Fonctions intégrées — numérique, conversion et représentation

ENGLISH	FRANÇAIS
What is the core idea of Built-in Functions — Numeric, Conversion & Representation? numeric and representation built-ins include abs(), round(), pow(), divmod(), int(), float(), complex(), bin(), hex(), oct(), repr() and format().	**Quelle est l'idée essentielle de « Fonctions intégrées — numérique, conversion et représentation » ?** Ce chapitre présente fonctions intégrées — numérique, conversion et représentation. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 65, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 65, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
raw = "0x1A"
value = int(raw, 16)
print(hex(value), abs(-value), round(3.14159, 3))
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 66 — Built-in Functions — Collections, Iteration & Inspection

Chapitre 66 — Fonctions intégrées — collections, itération et inspection

ENGLISH	FRANÇAIS
What is the core idea of Built-in Functions — Collections, Iteration & Inspection? collection and inspection built-ins include len(), iter(), next(), enumerate(), zip(), sorted(), min(), max(), sum(), all(), any(), isinstance() and related helpers.	**Quelle est l'idée essentielle de « Fonctions intégrées — collections, itération et inspection » ?** Ce chapitre présente fonctions intégrées — collections, itération et inspection. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 66, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 66, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
for index, value in enumerate(samples):
    print(index, value)
passed = all(v <= limit for v in samples)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 67 — Built-in Functions — Runtime, Objects & Dynamic Operations

Chapitre 67 — Fonctions intégrées — exécution, objets et opérations dynamiques

ENGLISH	FRANÇAIS
What is the core idea of Built-in Functions — Runtime, Objects & Dynamic Operations? runtime and dynamic built-ins include id(), type(), object(), getattr(), setattr(), hasattr(), callable(), globals(), locals(), compile(), eval() and exec(); dynamic execution requires strict trust boundaries.	**Quelle est l'idée essentielle de « Fonctions intégrées — exécution, objets et opérations dynamiques » ?** Ce chapitre présente fonctions intégrées — exécution, objets et opérations dynamiques. Il relie la sémantique Python aux choix de conception, à la validation des données et aux applications d'ingénierie.
Why does it matter in engineering software? It affects correctness, data interchange, performance and maintainability. In test, automation and hardware-facing code, explicit representation choices reduce ambiguous conversions and hidden state changes.	**Pourquoi est-ce important en logiciel d'ingénierie ?** Cela influence la justesse, l'échange des données, les performances et la maintenabilité. Dans les logiciels de test, d'automatisation et d'interface matériel, des choix explicites réduisent les conversions ambiguës et les changements d'état cachés.
What should be checked during design and review? Check type/representation assumptions, boundary cases, mutability or lifetime where relevant, conversion behavior, and whether the chosen API communicates intent clearly. Add tests around limits and malformed external data.	**Que faut-il vérifier pendant la conception et la revue ?** Vérifier les hypothèses de type et de représentation, les cas limites, la mutabilité ou la durée de vie lorsque pertinent, les conversions et la clarté de l'API. Ajouter des tests aux limites et avec des données externes malformées.

Engineering Notes / Notes d'ingénierie

ENGLISH	FRANÇAIS
Design rule Prefer explicit, readable operations over clever implicit conversions. For Chapter 67, document the expected input representation and failure behavior.	**Règle de conception** Privilégier des opérations explicites et lisibles plutôt que des conversions implicites astucieuses. Pour le chapitre 67, documenter la représentation d'entrée attendue et le comportement en cas d'échec.

Shared engineering example — code remains English only

```
if hasattr(device, "read") and callable(device.read):
    value = device.read()
# Never eval() untrusted instrument/user input.
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.